TUTORIAL VII

ROWS AND COLUMNS

The various *list* environments allows us to format some text into visually distinct *rows*. But sometimes the logical structure of the text may require these rows themselves to be divided into vertically aligned columns. For example, consider the material below typeset using the `\description` environment (doesn't it look familiar?)

Let's take stock of what we've learnt

Abiword A word processor

Emacs A text editor

TeX A typesetting program

A nicer way to typeset this is

Let's take stock of what we've learnt

AbiWord	A word processor
Emacs	A text editor
TeX	A typesetting program

Here the three *rows* of text are visually separated into two *columns* of left aligned text. This was produced by the `tabbing` environment in LaTeX.

VII.1. KEEPING TABS

VII.1.1. Basics

```
Let's take stock of what we've learnt
\begin{tabbing}
   \hspace{1cm}\= \textbf{AbiWord}\quad\= A word processor\\[5pt]
               \> \textbf{Emacs}       \> A text editor\\[5pt]
               \> \textbf{\TeX}        \> A typesetting program
 \end{tabbing}
```

Let's analyze it line by line. In the first line the first tab is put at a distance of 1 cm. from the left margin so that the text following it ('AbiWord' in boldface roman) starts from this point. The second tab is put at a distance of one `\quad` (this is an inbuilt length specification in TeX roughly equal to one space) after the word 'Abiword' in boldface roman so that the text following it ('A word processor' in ordinary roman face) start from this point. The `\\[5pt]` command signifies the end of the first line and also asks for a vertical space of 5 points between the first and the second lines. In the second line,

the first \> command makes the text following it ('Emacs' in boldface roman) to start from the first tab (already set in the first line), namely, 1 cm. from the left margin. The second \> line makes the text following it ('A text editor' in ordinary roman face) at the second tab already set, namely at a distance 1 cm plus *the length of the word 'AbiWord' in boldface roman* plus a \quad. The third line follows suit. The picture below will make this clear.

left margin

tab 1 *tab* 2
↓ ↓
AbiWord A word processor
Emacs A text editor
TEX A typesetting program

One should be careful in setting tabs. For example to typeset

TEX A typesetting program
Emacs A text editor
AbiWord A word processor

if you type

```
\begin{tabbing}
  \textbf{\TeX}\quad\= A typesetting program\\[5pt]
  \textbf{Emacs}\quad\> A text editor\\[5pt]
  \textbf{AbiWord}\quad\> A word processor
\end{tabbing}
```

then you end up with the output

TEX A typesetting program
Emacs A text editor
AbiWord A word processor

Do you see what happened? The first line set the first tab (the only tab in this example) at a distance of the length of the word 'TEX' in boldface roman plus a 'quad' from the left margin and the \> command in the second line makes the text following to atart from this tab, which is right next to the word 'Emacs' in this line. the same thing happens in the third line, which is worse, since the *position* of the tab is at the 'o' of 'AbiWord' and the next word 'A word processor' starts from this point, *and overwrites the previous word*. The correct way to obtain the output we want is to use a *dummy line* to mark the tabs, without actually typesetting that line. This is achieved by the \kill command in the tabbing environment, as shown below

```
\begin{tabbing}
  \textbf{AbiWord}\quad\= A word processor\kill
  \textbf{\TeX}\quad   \> A typesetting program\\[5pt]
```

```
  \textbf{Emacs}\quad  \> A text editor\\[5pt]
  \textbf{AbiWord}\quad\> A word processor
\end{tabbing}
```

New tabs, in addition to the ones already set by the first line (dummy or otherwise), can be set in any subsequent line. Thus the output

TEX : A typesetting program
Emacs : A text editor
a programming environment
a mail reader
and a lot more besides
AbiWord : A word processor

is obtained from the source

```
\begin{tabbing}
  \textbf{AbiWord}\quad\= : \= A word processor\kill\\
  \textbf{\TeX}\quad   \> : \> A typesetting program\\[5pt]
  \textbf{Emacs}\quad  \> : \> A text editor\\[5pt]
                       \>   \> \quad\= a programming environment\\[5pt]
                       \>   \>      \> a mail reader\\[5pt]
                       \>   \>      \> and a lot more besides\\[5pt]
  \textbf{AbiWord}\quad\> : \> A word processor
\end{tabbing}
```

Here the first line sets two tabs and the fourth line sets a third tab *after* these two. All the three tabs can then be used in the subsequent lines. New tab positions which *change* the ones set up by the first line, can also be introduced in any line by the \= command. Thus we can produce

Program : TEX
Author : Donald Knuth
Manuals :

Title	Author	Publisher
The TEXBook	Donald Knuth	Addison-Wesley
The Advanced TEX Book	David Salomon	Springer-Verlag

by the input

```
\begin{tabbing}
 Program\quad \= : \= \TeX\\[5pt]
 Author       \> : \> Donald Knuth\\[5pt]
 Manuals      \> :\\
 \quad\= The Advanced \TeX\ Book\quad\= David Salomon\quad
                                     \= Springer-Verlag\kill\\
 \>\textsf{Title}          \>\textsf{Author} \>\textsf{Publisher}\\[8pt]
```

```
  \>The \TeX Book           \>Donald Knuth    \>Addison-Wesley\\[5pt]
  \>The Advanced \TeX\ Book \>David Salomon   \>Springer-Verlag
\end{tabbing}
```

Here the first line sets teo tabs and the next two lines use these tabs. The third line sets three new tabs which *replace* the original tab positions. The next three lines use these new tab positions.

VII.1.2. Pushing and popping

What if you change the tab positions and then want the original settings back? Here's where the command pair `\pushtabs ... \poptabs` ia useful. Thus to typeset

Program : TEX
Author : Donald Knuth
Manuals :

Title	Author	Publisher
The TEXBook	Donald Knuth	Addison-Wesley
The Advanced TEX Book	David Salomon	Springer-Verlag

Tutorial : http://tug.org.in/tutorial

we type

```
\begin{tabbing}
 Program\quad \= : \= \TeX\\[5pt]
 Author       \> : \> Donald Knuth\\[5pt]
 Manuals      \> :\\
 \pushtabs
 \quad\= The Advanced \TeX\ Book \quad \= David Salomon \quad
      \= Springer-Verlag\kill\\
 \>\textsf{Title}           \>\textsf{Author} \>\textsf{Publisher}\\[8pt]
 \>The \TeX Book            \>Donald Knuth    \>Addison-Wesley\\[5pt]
 \>The Advanced \TeX\ Book  \>David Salomon   \> Springer-Verlag\\[8pt]
 \poptabs
 Tutorial                   \> :              \> "http://tug.org.in/tutorial"
\end{tabbing}
```

Here the first three lines follow a tabbing scheme, the next three lines follow another tabbing scheme and the last line reverts back to the original scheme. Here the `\pushtabs` command stores the current tabbing scheme and removes it so that a new tabbing scheme can be set up; and the `\poptabs` commands reactivates the original scheme. These commands can be nested.

VII.1.3. More commands

There are some more useful commands available in the `tabbing` environment. The `\+` command *given at the end of a line* makes *every subsequent line* start at the first tab; with `\+\+` at the end of a line, all subsequent lines start at the second tab and so on. The effect of each `\+` can be neutralized by one `\-` command at the end of a line. The

command \< *at the beginning of a line* neutralizes the effect of one \+ command for that particular line.

The command \‘ (left quote) puts the text following flush right against the right margin. Naturally we cannot use a \= or \> after this in a line.

Another interesting command is \’ (right quote). Within the `tabbing` environment an input of the form `left_text\’right_text` puts the `right_text` at the current tab and the `left_text` just before this tab with a bit of spacing (preassigned by the parameter `\tabbingsep`).

The example below illustrates all the tabbing commands we’ve discussed

```
\begin{tabbing}
 Row 1 Column 1\hspace{2cm}
                    \= Row 1 Column 2\\[5pt]
               \> Row 2 Column 2\hspace{1.5cm}\=Row 2 Column 3\+\+\\[5pt]
                                                 Row 3 Column 3\-\\[5pt]
                  Row 4 Column 2              \>Row 4 Column 3\\[5pt]
 \< Row 5 Column 1  \> Row 5 Column 2         \>Row 5 Column 3\\[5pt]
                       Row 6 Column 2         \>Row 6 Column 3\-\\[5pt]
 Row 7 Column 1     \> Row 7 Column 2         \>Row 7 Column 3\\[5pt]
 Row 8 Column 1                                  \‘Right\\[5pt]
 Row 9 Column 1     \> and\’Row 9 Column 2\\[5pt]
 \pushtabs
 \quad\= Row 10 New Column 1\hspace{2.5cm}\= Row 10 New Column 2\\[5pt]
      \> Row 11 New Column 2              \> Row 11 New Column 2\\[5pt]
 \poptabs
 Row 12 Old Column 1\> Row 12 Old Column 2\>Row 12 Old Column 3
\end{tabbing}
```

It produces the following output

Row 1 Column 1 Row 1 Column 2
Row 2 Column 2 Row 2 Column 3
Row 3 Column3
Row 4 Column 2 Row 4 Column 3
Row 5 Column 1 Row 5 Column 2 Row 5 Column 3
Row 6 Column 2 Row 6 Column 3
Row 7 Column 1 Row 7 Column 2 Row 7 Column 3
Row 8 Column 1 Right
Row 9 Column 1 and Row 9 Column 2
Row 10 New Column 1 Row 10 New Column 2
Row 11 New Column 2 Row 11 New Column 2
Row 12 Old Column 1 Row 12 Old Column 2 Row 12 Old Column 3

Recall that the commands \=. \‘ and \’ are used for various accents outside the `tabbing` environment. If these are needed within the `tabbing` environment, they can be produced with the commands \a=. \a‘ or \a’ commands.

One final word. You might’ve noted in the examples above that we give a sort of